PLAN DE PRUEBAS

TuHospedaje

Plan y Reporte de Pruebas de Software — Sprint 3

TuHospedaje — Búsqueda, Disponibilidad, Favoritos y Contenido

62 de 62 verificados
ESCENARIOS FUNCIONALES

139 tests de backend
AUTOMATIZADAS

TC-30 a TC-34
MEJORAS COMPLEMENTARIAS

Equipo de Desarrollo · Mayo 2026

Foco del Incremento: Búsqueda por ciudad y fechas, Disponibilidad y reservas, Favoritos, Políticas, Reseñas, Compartir en redes **Enfoque de Testing:** Pruebas de API (Postman + automatizadas con MockMvc/Testcontainers), Verificación de UI Manual

1. Matriz Detallada de Casos de Prueba (Test Cases)

TC-22: Módulo de Búsqueda (US #22)

- **Historias de Usuario Asociadas:** US #22 (Realizar búsqueda)
- **Precondiciones:** Servidor backend activo. Al menos 3 alojamientos en BD en diferentes ciudades.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (JUnit 5 + MockMvc + Testcontainers).

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/lodgings/search?city=buenos` (público)	HTTP 200. Lista filtrada por ciudad (case-insensitive)	✔ Pasa
2	`GET /api/lodgings/search` sin parámetros	HTTP 200. Todos los alojamientos	✔ Pasa
3	`GET /api/lodgings/search?guests=4`	HTTP 200. Solo alojamientos con maxGuests >= 4	✔ Pasa
4	`GET /api/lodgings/search?minPrice=100&maxPrice=200`	HTTP 200. Solo alojamientos en rango de precio	✔ Pasa
5	`GET /api/lodgings/search?checkIn=2026-07-01&checkOut=2026-07-05`	HTTP 200. Excluye alojamientos con reservas solapadas	✔ Pasa
6	`GET /api/lodgings/cities?q=bu` (público)	HTTP 200. Lista de ciudades que coinciden	✔ Pasa
7	`GET /api/lodgings/cities?q=` (vacío)	HTTP 200. Todas las ciudades	✔ Pasa
8	Búsqueda sin resultados	HTTP 200. Array vacío `[]`	✔ Pasa
9	`GET /api/lodgings/search?checkIn=invalido`	HTTP 400 Bad Request	✔ Pasa

TC-23: Módulo de Disponibilidad y Reservas (US #23)

- **Historias de Usuario Asociadas:** US #23 (Visualizar disponibilidad)
- **Precondiciones:** Alojamiento existente en BD. Usuario autenticado con JWT válido.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/lodgings/{id}/availability?checkIn=...&checkOut=...` (público)	HTTP 200. `{ available: true/false }`	✔ Pasa

2	`POST /api/reservations` con datos válidos y token	HTTP 201. Reserva creada con status CONFIRMED	✔ Pasa
3	`POST /api/reservations` en fechas ya ocupadas	HTTP 409 Conflict	✔ Pasa
4	`POST /api/reservations` con checkOut anterior a checkIn	HTTP 400 Validation Error	✔ Pasa
5	`POST /api/reservations` con fecha pasada	HTTP 400 Validation Error	✔ Pasa
6	`POST /api/reservations` sin token JWT	HTTP 401 Unauthorized o 403 Forbidden	✔ Pasa
7	`GET /api/reservations/{id}` con token válido	HTTP 200. Detalle de reserva	✔ Pasa
8	`GET /api/reservations/{id}` con ID inexistente	HTTP 404 Not Found	✔ Pasa
9	Dos reservas simultáneas mismo alojamiento/fechas	Una HTTP 201, la otra HTTP 409 (OptimisticLock)	✔ Pasa

TC-24/25: Módulo de Favoritos (US #24, #25)

- **Historias de Usuario Asociadas:** US #24 (Marcar favorito), US #25 (Listar favoritos)
- **Precondiciones:** Usuario autenticado. Alojamiento existente.
- **Tipos de Verificación:** API Rest, UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/favorites/{lodgingId}` con token	HTTP 200. Favorito agregado	✔ Pasa
2	`GET /api/favorites` con token	HTTP 200. Lista de favoritos del usuario	✔ Pasa
3	`DELETE /api/favorites/{lodgingId}` con token	HTTP 200. Favorito eliminado	✔ Pasa
4	`POST /api/favorites/{lodgingId}` sin token	HTTP 401/403	✔ Pasa
5	Corazón visible en ProductCard (autenticado)	Se muestra icono ♥	✔ Pasa
6	Corazón NO visible en ProductCard (anónimo)	No se muestra icono	✔ Pasa
7	Click corazón agrega/quita favorito	Toggle visual + llamada API	✔ Pasa

TC-26: Módulo de Políticas (US #26)

- **Historias de Usuario Asociadas:** US #26 (Ver bloque de políticas)
- **Precondiciones:** Políticas creadas en BD. Alojamiento con políticas asociadas.

- **Tipos de Verificación:** API Rest, UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/policies` (público)	HTTP 200. Lista de políticas	✔ Pasa
2	`POST /api/policies` con token ADMIN	HTTP 201. Política creada	✔ Pasa
3	`POST /api/policies` sin token ADMIN	HTTP 403 Forbidden	✔ Pasa
4	Sección de políticas en ProductDetail	Título subrayado, columnas con ícono + título + descripción	✔ Pasa

Cobertura automatizada (frontend) — agregada en auditoría posterior: el paso 4 no tenía assertion automatizada hasta entonces (solo verificación manual). `ProductDetail.test.jsx` — `describe('ProductDetail - Policies detail', ...)` ahora verifica que las políticas reales del alojamiento se rendericen por nombre y descripción (commit `2bced9d`).

TC-27: Módulo de Compartir (US #27)

- **Historias de Usuario Asociadas:** US #27 (Compartir en redes sociales)
- **Precondiciones:** Alojamiento visible en ProductDetail.
- **Tipos de Verificación:** UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Click botón "Compartir" en ProductDetail	Pop-up con opciones Facebook, Twitter, WhatsApp	✔ Pasa
2	Pop-up muestra imagen + descripción + enlace	Contenido visible y correcto	✔ Pasa
3	Click en red social	Abre nueva pestaña con URL de compartir	✔ Pasa

Cobertura automatizada (frontend) — agregada en auditoría posterior: los pasos 1-2 se verificaban antes solo comprobando que el modal abría, sin verificar su contenido real (test con `vi.mock` que stubeaba `ShareModal`). `ProductDetail.test.jsx` — `describe('ProductDetail - ShareModal', ...)` ahora verifica la URL y descripción realmente renderizadas contra el componente real (commit `2bced9d`).

Comportamiento del mensaje para compartir: cuando el alojamiento tiene descripción, el modal la muestra y la agrega al mensaje inicial editable, quitando espacios al principio y al final. Twitter y WhatsApp reciben ese mensaje; Instagram copia el mensaje y el enlace para completar la publicación manualmente. Si la descripción falta o contiene solo espacios, se conserva el mensaje anterior con el nombre. Facebook y

“Copiar enlace” siguen usando únicamente la URL. `ShareModal.test.jsx` cubre estos contratos locales; esto no prueba publicación ni entrega en los proveedores externos.

TC-28: Módulo de Reseñas (US #28)

- **Historias de Usuario Asociadas:** US #28 (Puntuar producto)
- **Precondiciones:** Alojamiento existente. Usuario autenticado.
- **Tipos de Verificación:** API Rest, UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/ratings` con token y datos válidos	HTTP 201. Reseña creada	✔ Pasa
2	`POST /api/ratings` sin token	HTTP 401/403	✔ Pasa — corregido, ver nota
3	`GET /api/ratings/lodging/{id}` (público)	HTTP 200. Lista de reseñas con promedio	✔ Pasa — corregido, ver nota
4	Formulario de reseña visible solo para autenticados	Selector de estrellas + textarea	✔ Pasa
5	Promedio de estrellas se actualiza al agregar reseña	Cálculo dinámico correcto	✔ Pasa
6	Lista de reseñas muestra nombre, fecha, estrellas, comentario	Datos visibles y formateados	✔ Pasa

Corrección — auditoría posterior encontró un defecto real, no solo un gap de test: los pasos 2 y 3 estaban marcados "✔ Pasa" sin ningún test a nivel HTTP que los probara — solo existían tests de `RatingServiceImplTest` con el repositorio mockeado, que no pasan por la cadena de filtros de Spring Security. Al agregar la cobertura HTTP real (`RatingControllerIntegrationTest`), el paso 3 **falló**: `SecurityConfig.java` nunca tuvo la regla `permitAll()` para `GET /api/ratings/**` (a diferencia de `/api/categories` , `/api/features` , `/api/policies` y `/api/lodgings` , que sí la tienen), así que el endpoint documentado como público en `RatingController` en realidad exigía autenticación. Impacto real: cualquier visitante anónimo en la ficha de un alojamiento recibía un 403 silencioso — `ReviewsSection.jsx` lo atrapa con `.catch(() => {})` y renderiza "0.0 ★ (0 reseñas)" sin ningún error visible, indistinguible de "sin reseñas todavía". Nunca se detectó en QA manual por eso mismo.

Fix: agregada `.requestMatchers(HttpMethod.GET, "/api/ratings/**").permitAll()` en `SecurityConfig.java` . Tests nuevos en `RatingControllerIntegrationTest` : `shouldReturnForbiddenWhenCreatingRatingWithoutToken` (paso 2, POST sin token → 403) y `shouldReturnRatingsForLodgingWithoutAuthentication` (paso 3, GET sin ninguna cookie → 200 con el payload real). Commit `30caab9` .

TC-29: Módulo de Eliminar Categoría (US #29)

- **Historias de Usuario Asociadas:** US #29 (Eliminar categoría)
- **Precondiciones:** Categorías existentes en BD. Usuario ADMIN autenticado.
- **Tipos de Verificación:** UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Click "Eliminar" en categoría	ConfirmDialog con mensaje descriptivo	✔ Pasa
2	Click "Confirmar" en diálogo	Categoría eliminada, tabla refrescada	✔ Pasa
3	Click "Cancelar" en diálogo	Diálogo cerrado, categoría intacta	✔ Pasa

TC-30: Galería de imágenes con modal viewer (UI Manual)

- **Historias de Usuario Asociadas:** Mejora complementaria
- **Precondiciones:** ProductDetail con al menos 1 imagen
- **Tipos de Verificación:** UI Manual

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Click en imagen principal	Modal lightbox se abre con imagen completa	✔ Pasa
2	Click flecha derecha / tecla `→`	Avanza a la siguiente imagen	✔ Pasa
3	Click flecha izquierda / tecla `←`	Retrocede a la imagen anterior	✔ Pasa
4	Presionar tecla `Esc` / click overlay	Modal se cierra	✔ Pasa
5	Click en miniatura	Modal se abre desde esa imagen	✔ Pasa
6	Verificar contador	Muestra `N / M` correcto	✔ Pasa

TC-31: Íconos Lucide en features/policies (UI Manual)

- **Historias de Usuario Asociadas:** Mejora complementaria
- **Precondiciones:** Alojamiento con features y policies cargadas
- **Tipos de Verificación:** UI Manual

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO	ESTADO
1	ProductDetail: features	SVGs Lucide visibles, no texto CSS	✔ Pasa
2	ProductDetail: policies	Ícono SVG + nombre	✔ Pasa
3	Admin → Features/Policies	Tabla muestra SVG + key	✔ Pasa

4	Admin → Crear con IconPicker	Grilla con búsqueda y preview funcional	✔ Pasa

TC-32: Admin rediseñado + route guard (UI Manual)

- **Historias de Usuario Asociadas:** Mejora complementaria
- **Precondiciones:** Usuario ADMIN autenticado
- **Tipos de Verificación:** UI Manual

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO	ESTADO
1	Navegar a `/admin`	Sidebar + topbar + dashboard. Header público oculto	✔ Pasa
2	Click en stat card	Navega al tab correspondiente	✔ Pasa
3	`/admin` sin login	Redirige a `/login`	✔ Pasa
4	`/admin` como user normal	Redirige a `/`	✔ Pasa
5	FAB en admin tabs	Abre modal de creación	✔ Pasa

TC-33: Favoritos sincronizados (UI Manual)

- **Historias de Usuario Asociadas:** US #24/#25
- **Precondiciones:** Usuario logueado con al menos 1 favorito
- **Tipos de Verificación:** UI Manual

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO	ESTADO
1	Home: corazones cargados	Favoritos existentes aparecen llenos	✔ Pasa
2	Click corazón (quitar)	Se vacía sin recargar	✔ Pasa
3	Click corazón (agregar)	Se llena sin recargar	✔ Pasa
4	SearchResults: mismo estado	Coincide con Home	✔ Pasa

TC-34: CategoryCard (UI Manual)

- **Historias de Usuario Asociadas:** Mejora complementaria
- **Precondiciones:** Categorías cargadas en BD
- **Tipos de Verificación:** UI Manual

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO	ESTADO
1	Home: categorías	Cards con ícono + nombre + descripción	✔ Pasa
2	Click en CategoryCard	Filtra alojamientos por categoría	✔ Pasa

2. Resumen de Ejecución

TIPO DE PRUEBA	CANTIDAD	ESTADO
Tests Automatizados Backend (JUnit 5 + MockMvc)	139 tests	✔ Todos pasan
Casos de Prueba Funcionales (Plan)	62 escenarios	✔ 62/62 verificados

3. Cobertura por Historia de Usuario

USER STORY	CANTIDAD TC	ESTADO
US #22 — Búsqueda	9 TC	✔ Automatizado
US #23 — Disponibilidad	9 TC	✔ Automatizado
US #24/#25 — Favoritos	7 TC	✔ Automatizado + Manual
US #26 — Políticas	4 TC	✔ Automatizado + Manual
US #27 — Compartir	3 TC	✔ Manual
US #28 — Reseñas	6 TC	✔ Automatizado + Manual
US #29 — Eliminar categoría	3 TC	✔ Manual
Galería (TC-30)	6 TC	✔ Manual
Lucide Icons (TC-31)	4 TC	✔ Manual
Admin rediseñado (TC-32)	5 TC	✔ Manual
Favoritos sincronizados (TC-33)	4 TC	✔ Manual
CategoryCard (TC-34)	2 TC	✔ Manual

4. Herramientas Utilizadas

HERRAMIENTA	PROPÓSITO
JUnit 5 + Mockito	Tests unitarios de servicios
MockMvc + Testcontainers	Tests de integración con MariaDB efímera
Postman	Pruebas manuales de API
Navegador (Chrome)	Verificación de UI
Swagger UI	Documentación y exploración de endpoints

5. Defectos Encontrados

ID	DESCRIPCIÓN	SEVERIDAD	ESTADO
—	Ningún defecto crítico encontrado durante el Sprint 3 original	—	—
BUG-06 (encontrado en auditoría posterior)	`GET /api/ratings/lodging/{id}` (US #28) estaba documentado como público pero `SecurityConfig` nunca tuvo la regla `permitAll()` correspondiente, exigiendo autenticación en la práctica. Visitantes anónimos recibían un 403 silenciado por `ReviewsSection.jsx` (`.catch(() => {})`), viendo "0.0 ★ (0 reseñas)" sin error visible. No detectado en el sprint original por falta de test a nivel HTTP (ver TC-28).	Media (visibilidad de contenido público)	✔ Corregido — commit `30caab9`